DragonflyDB Shared-Nothing Transactions Breakdown

Dissection: Arpit Bhayani & Roman Gershman
Focus: Very Lightweight Locking (VLL) & Fibers

1. Executive Summary & Problem Statement

In modern in-memory data store architecture, achieving multi-million operations per second requires scaling across dozens of CPU cores. The gold standard for multicore hardware is the **shared-nothing, thread-per-core model**: the global keyspace is partitioned into independent shards, where each shard is managed exclusively by a dedicated CPU thread.

Under this model, single-key operations (`GET` , `SET` , `INCR`) execute with zero lock contention, zero mutex overhead, and zero CPU cache-coherency ping-pong traffic across socket buses.

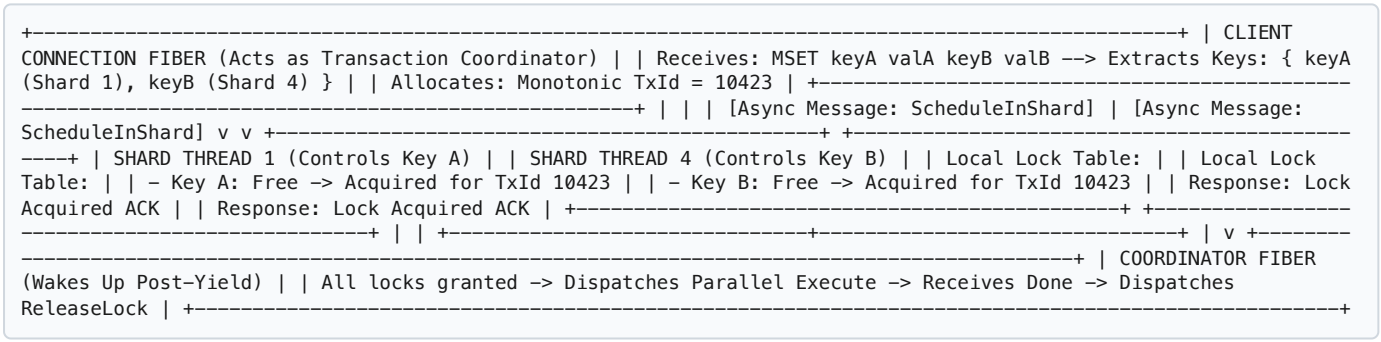
> **The Multi-Key Conflict:** While shared-nothing architecture trivializes single-key queries, it creates severe complexity for **multi-key transactional commands** (such as `MSET keyA valA keyB valB` , `MULTI/EXEC` blocks, and complex Lua scripts). When operations touch keys scattered across disparate CPU threads, conventional systems must either restrict operations, employ coarse-grained global locks, or rely on complex distributed 2-Phase Locking (2PL) that stalls worker threads.

2. The Redis Dichotomy: Standalone vs. Redis Cluster

- **Redis Standalone:** Constrains execution to a single OS thread. Multi-key operations are inherently atomic because the single thread processes commands sequentially. However, this caps throughput to a single CPU core ($ pprox 100 ext{k} - 150 ext{k} ext{ QPS}$), leaving remaining server cores idle.
- **Redis Cluster:** Distributes keys across 16,384 hash slots across independent nodes. Crucially, Redis Cluster **forbids multi-key transactions across different slots**, throwing a fatal `CROSSSLOT Keys in request don't hash to the same slot` error. Engineers must manually force keys into identical slots via Hash Tags (e.g. `{user100}:profile` , `{user100}:orders`), which induces catastrophic hot-slot imbalances under heavy write patterns.

3. The DragonflyDB Paradigm: Cross-Shard VLL Message Passing

DragonflyDB bridges this fundamental trade-off: it delivers multi-core vertical throughput ($4 ext{M} - 6.4 ext{M+ QPS}$) while providing **fully atomic, strictly serializable multi-key transactions across arbitrary shards** without requiring Hash Tags.



4. Very Lightweight Locking (VLL) Algorithm Adaptation

DragonflyDB adapts the **Very Lightweight Locking (VLL)** algorithm (Ren et al., VLDB 2012) for multicore in-memory execution. Rather than managing complex distributed transaction managers or heavyweight lock records, VLL represents locks as lightweight counter flags and local wait queues embedded directly within shard runtimes.

- **Intent Lock Types:** Operations declare lock intent prior to scheduling:
  - `LockType::READ`: Shared intent lock for read-only queries (e.g. `MGET`, `SUNION`). Multiple read transactions can access the same key concurrently.
  - `LockType::WRITE`: Exclusive intent lock for mutating commands (e.g. `MSET`, `SMOVE`, `MULTI/EXEC`). Blocks all concurrent reads and writes on that key.
- **Shard-Local Lock Queues:** Every shard maintains a hash map of active locks and FIFO pending queues. If a requested key is already locked by a prior transaction, the arriving transaction is placed into that shard's local wait list.

5. Cooperative Fibers: Non-Blocking Context Switching

The defining breakthrough in DragonflyDB's transactional efficiency is the integration of VLL with **user-space cooperative fibers** (powered by the Helio library).

> **The Problem with OS Threads:** In standard multi-threaded databases, if a thread attempts to acquire a lock held by another thread, it blocks (via `pthread_mutex_lock` or condition variables). This invokes the OS kernel scheduler, incurs expensive context-switching latency ($1 - 2\mu ext{s}$), and idles the CPU core.

In DragonflyDB:

1. Every client connection is backed by an independent fiber.
2. When a transaction coordinator fiber dispatches asynchronous `ScheduleInShard` messages across multiple shards, it **yields execution** to the Helio runtime scheduler.
3. While waiting for cross-shard lock ACKs, the underlying OS thread remains 100% busy servicing other client fibers that are ready to run!
4. Once all shard threads acknowledge lock acquisition, the Helio scheduler wakes up the coordinator fiber, which immediately resumes execution.

6. Monotonic TxId Ordering & Mathematical Deadlock Freedom

In distributed systems, multi-resource locking frequently results in cyclic deadlocks (e.g. Tx 1 locks Key A and waits for Key B; Tx 2 locks Key B and waits for Key A).



DragonflyDB guarantees total deadlock freedom without running background deadlock detectors:

- **Global Monotonic Counter:** Every multi-key transaction is stamped with a monotonically increasing sequence identifier (`TxId`).
- **Deterministic Total Ordering:** Across all shards, transactions are queued and granted locks in exact `TxId` order. Because resource acquisition follows a strict global sequence, cyclic dependencies are mathematically impossible ($T_1 < T_2 \implies T_2 ext{ never holds a lock required by } T_1$).

7. Multi-Key Transaction Execution Lifecycle

A multi-key transaction transitions through five deterministic phases:

```
[Ingress Phase] Client sends multi-key command -> Coordinator fiber parses keys & maps to shards. | [Sequencing Phase] Monotonic TxId assigned; commands classified into Shared-Read or Exclusive-Write. | [Scheduling Phase] Coordinator dispatches async ScheduleInShard() to target shards and yields CPU. Each shard locks free keys or enqueues Tx in its local FIFO wait list. | [Execution Phase] All ACKs received -> Coordinator fiber wakes up -> Dispatches parallel execute. Shard threads execute mutations against their local memory concurrently. | [Release Phase] Coordinator receives completion ACKs -> Dispatches ReleaseLock to shards. Shards release locks and notify head of wait list -> Coordinator replies to Client.
```

8. Architectural Comparison: Concurrency & Transactions

Property	Redis Standalone	Redis Cluster	DragonflyDB
Concurrency Architecture	Single-threaded event loop	Independent Redis processes	Multi-threaded shared-nothing (fibers per core)
Multi-Key Operations	Atomic (limited to 1 CPU core)	**Strictly Forbidden** across slots	**Arbitrary Multi-Key Atomicity** across shards
Cross-Slot Constraint	None	Requires Hash Tags (e.g. `{tag}`)	None (automatic shard coordination)
Locking Mechanism	None (sequential execution)	None (disallowed across slots)	Fiber-based Very Lightweight Locking (VLL)
Deadlock Handling	Impossible (single thread)	N/A	Impossible via Monotonic `TxId` sorting
Single Node Throughput	$100 ext{k} - 150 ext{k} ext{ QPS}$	$100 ext{k} ext{ QPS}$ per node process	**$4 ext{M} - 6.4 ext{M+} ext{ QPS}$ on single server**

9. First-Principles Summary & Design Lessons

- **Decouple State Isolation from Data Sharing:** Shared-nothing architecture does not preclude cross-partition coordination. Asynchronous message passing combined with localized intent locking enables full atomicity without shared memory locks.
- **Fibers Eliminate Latency Penalties:** Synchronization overhead becomes negligible when lock waits yield control to ready user-space tasks rather than invoking kernel thread context switches.
- **Total Ordering Prevents Deadlocks:** Stamping transactions with a global monotonic sequence number guarantees deterministic lock resolution, rendering distributed deadlock detection obsolete.